

Count Pairs Such That $\text{nums}[i] * \text{nums}[j]$ is Divisible by k

Problem Statement

Given an integer array nums and an integer k , count the number of pairs (i, j) such that

- $0 \leq i < j < n$
 - $(\text{nums}[i] * \text{nums}[j]) \% k == 0$
-

First Thought (Wrong Approach)

Since the previous problem

$$(a+b)\%k==0$$

used remainders,

our first instinct is

"Can I again store remainders?"

Let's investigate.

Step 1 — Test the Remainder Idea

Suppose

$$k = 6$$

Current number

2

Its remainder is

2

Question

Which remainder should the second number have?

Let's test.

$$2 \times 3 = 6 \quad \checkmark$$

$$6 \% 6 = 0$$

Remainder

3

works.

Now test

$$2 \times 9 = 18 \quad \checkmark$$

$$18 \% 6 = 0$$

Remainder

3

works again.

Now try

$$2 \times 4 = 8$$

$$8 \% 6 = 2$$

Fails.

Remainder

4

fails.

Now

$$2 \times 10 = 20$$

$$20 \% 6 = 2$$

Again remainder

4

fails.

Now let's take another example.

$$k = 12$$

$$\text{Current} = 4$$

Need

$$4 \times x \text{ divisible by } 12$$

Possible partners

3

6

9

12

15

18

Their remainders are

3
6
9
0
3
6

Notice

Many different remainders work.

Unlike the addition problem,

there is **NO SINGLE complementary remainder**.

Important Observation

For

Addition

one remainder gives

one complement

For

Multiplication

one remainder may have

many valid partners.

Therefore

Remainder HashMap

✗ DOES NOT WORK

Step 2 — Ask a Better Question

Instead of asking

Which remainder do I need?

Ask

What is actually missing to make the product divisible by k?

This shifts our thinking completely.

Step 3 — Think in Terms of Prime Factors

Suppose

$$k = 12$$

$$12 = 2^2 \times 3$$

Current number

$$4$$

$$= 2^2$$

It already contributes

$$2^2$$

What's still missing?

$$3$$

So any number containing factor

3

will complete the product.

Notice

We are NOT thinking about remainders anymore.

We are thinking about

Prime Factors

Step 4 — How Do We Know What Factors a Number Already Has?

We need a function that tells us

"How much of k is already present inside this number?"

That function is

`gcd(number, k)`

because

`gcd(number, k)`

=

Common factors shared by
number and k

Example

`k = 12`

Current number

8

$$=2^3$$

Common factors with 12

$$\gcd(8, 12)=4$$

$$=2^2$$

Meaning

This number already contributes

$$2^2$$

Still missing

3

Exactly what we wanted.

Another example

$$\text{Current} = 18$$

$$18=2\times 3^2$$

$$\gcd(18, 12)=6$$

$$=2\times 3$$

Already contributes

$$2\times 3$$

Needs only

2

to complete 12.

Step 5 — Build the Condition

Suppose

$$g1 = \text{gcd}(a, k)$$

$$g2 = \text{gcd}(b, k)$$

Then

$a * b$ divisible by k

is equivalent to

$$(g1 * g2) \% k == 0$$

This is the key observation.

Instead of checking

$a * b$

we only need

gcd values.

Why Does This Work?

Remember


```
gcd(num, k)
```

contains every factor of k already present in that number.

If together

```
g1 × g2
```

contains every prime factor of k

then

```
product  
  
is divisible by k.
```

Dry Run

Input

```
nums = [2,2,1,7,5,3]  
  
k = 4
```

Initially

```
mp={}  
  
ans=0
```

Number = 2

```
gcd(2,4)=2
```

Previous gcds

none

Store

2 : 1

Number = 2

g=2

Check

$2 \times 2 = 4$

$4 \% 4 == 0$

Found

1 pair

Store

2 : 2

Answer

1

Number = 1

$\text{gcd}(1, 4) = 1$

Check

$$1 \times 2 = 2$$

Not divisible

Store

$$1 : 1$$

$$2 : 2$$

Number = 7

$$\text{gcd}(7, 4) = 1$$

Check

$$1 \times 1 = 1$$

×

$$1 \times 2 = 2$$

No pairs.

Store

$$1 : 2$$

$$2 : 2$$

Number = 5

$$\text{gcd} = 1$$

Again

No pair

Store

1:3

2:2

Number = 3

gcd=1

Again

No pair

Store

1:4

2:2

Final Answer

1

Only

$2 \times 2 = 4$

is divisible by 4.

Building the Optimized Logic

Instead of storing

Number

store

`gcd(number,k)`

For every new number

↓
Find gcd
↓
Compare with every previous gcd
↓
If
 $(gcd1 * gcd2) \% k == 0$
count it
↓
Store current gcd

Code

```
from math import gcd

def countPairs(nums, k):
    mp = {}
    ans = 0
    for num in nums:
        g = gcd(num, k)
        for prev_g in mp:
```

```
        if (g * prev_g) % k == 0:
            ans += mp[prev_g]

    mp[g] = mp.get(g, 0) + 1

    return ans
```

Complexity

Suppose

d = Number of divisors of k

Important fact

$\text{gcd}(\text{num}, k)$
is ALWAYS
a divisor of k

Therefore

Maximum hashmap size

d

Hence

Time
 $O(n \times d)$

Space

$O(d)$

Since d is usually small, this is much faster than $O(n^2)$.

Why Not Store Numbers?

Because

Numbers

↓

Too many values

But

$\text{gcd}(\text{num}, k)$

↓

Only divisors of k

Much smaller search space.

Comparison with Previous Problem

Sum Divisible	Product Divisible
Store remainder	Store gcd
One complement exists	Many partners exist
Complement formula	Compare gcd products
$O(n)$	$O(n \times \text{divisors}(k))$

Pattern Recognition

Sum Problem

$(a+b) \% k == 0$

Think

Modulo

↓

Remainder

↓

Complement

↓

HashMap

Product Problem

$(a*b)\%k==0$

Think

Prime Factors

↓

GCD

↓

Divisors

↓

HashMap on gcd values

Interview Thinking Framework

Whenever an interviewer gives a condition,

don't immediately jump into coding.

Ask

Step 1

What mathematical operation is involved?

+

-

*

/

XOR

GCD

LCM

Step 2

Can I transform the numbers into a smaller representation?

Examples

Remainder

Prefix Sum

Parity

Difference

GCD

Prime Factors

Step 3

Can I search using that transformed representation instead of the original numbers?

If yes,

HashMap is usually possible.

Recognition Cheat Sheet

Condition Involves	First Thing to Think
$a + b$	Remainder / Complement / Prefix Sum
$a - b$	Difference / Complement
$a \text{ XOR } b$	Bit properties / Trie / XOR Prefix
$a * b$	Prime Factors / GCD / Divisors
LCM / GCD	Number Theory / Prime Factorization
Prefix condition	Prefix Sum / Prefix XOR
Sliding window keywords ("longest", "smallest", "at most", "exactly")	Sliding Window / Two Pointers

Master Pattern Sheet

Condition involves

$a + b$

—————> Try remainder/complement.

$a - b$

—————> Difference/complement.

$a \text{ XOR } b$

—————> Bit properties / Trie / XOR prefix.

$a * b$

—————> Think prime factors, GCD, divisors.

LCM/GCD

—————> Number theory instead of simple modulo.

Quick Revision Sheet

Sum Divisible

Store

$\text{num} \% k$

Need

$$(k - r) \% k$$

Complexity

$$O(n)$$

Product Divisible

Store

$$\text{gcd}(\text{num}, k)$$

Need

$$(g1 * g2) \% k == 0$$

Complexity

$$O(n \times \text{divisors}(k))$$

Final Memory Trick

Addition problems usually become Complement + HashMap. Multiplication problems usually become Prime Factors/GCD + Divisors.

Never memorize the formula.

Instead ask:

"What mathematical information determines whether the condition becomes true?"

- For **addition**, it's the **remainder**.
- For **multiplication**, it's the **prime factors already contributed toward k** , captured by $\text{gcd}(\text{num}, k)$.